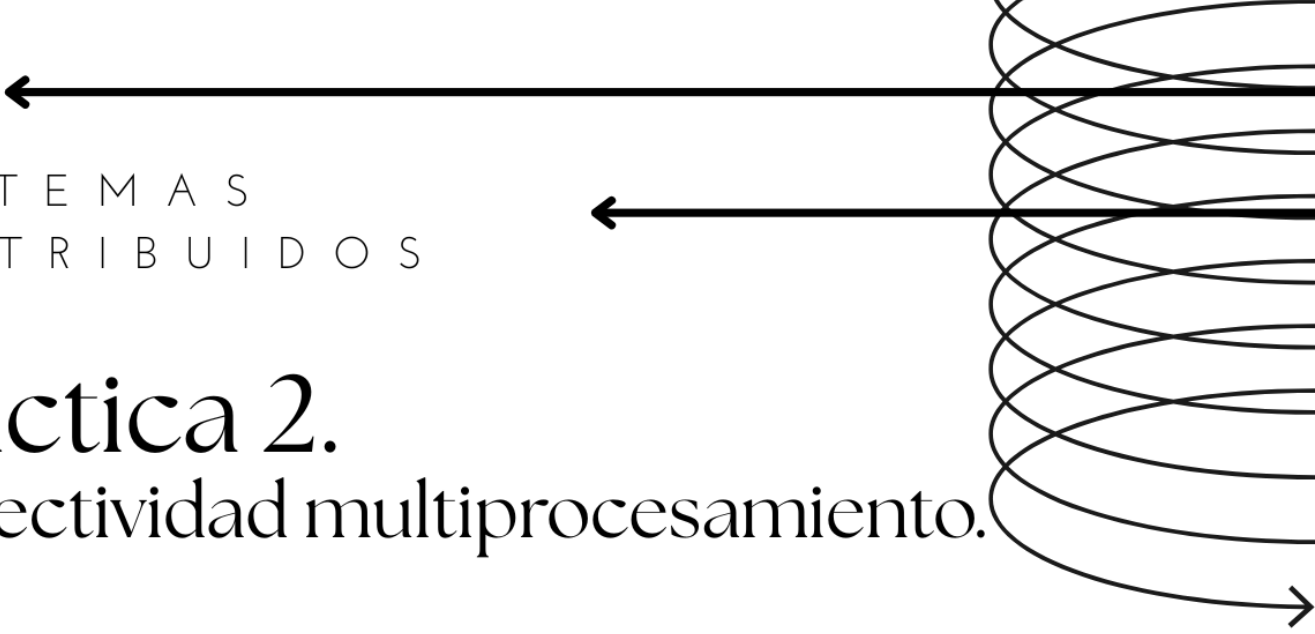




SISTEMAS
DISTRIBUIDOS

Práctica 2.

Conectividad multiprocesamiento.

MICHELLE ANGUIANO RODEA

ESCOM

PROFESOR CHADWICK CARRETO
ARELLANO

GRUPO: 7CM6



Instituto
politécnico
nacional



ESCOM



1. Antecedentes (Introducción)

En el ámbito de los sistemas distribuidos y de redes, la **comunicación entre procesos** es un pilar fundamental para el funcionamiento de aplicaciones modernas. Desde servicios web, sistemas de mensajería, hasta plataformas de streaming, todos dependen del modelo **cliente-servidor**, que permite a múltiples dispositivos o programas intercambiar información a través de una red.

La herramienta más utilizada para implementar esta comunicación son los **sockets**. Un socket es un punto final en una conexión de red que permite enviar y recibir datos. En el caso de la comunicación TCP (Transmission Control Protocol), los sockets garantizan que los mensajes lleguen de forma ordenada, sin pérdidas y sin duplicaciones, lo que los hace ideales para aplicaciones que requieren confiabilidad.

El modelo cliente-servidor funciona de la siguiente manera:

- **Servidor:** Es un proceso que permanece en ejecución de forma indefinida, a la espera de conexiones entrantes en un puerto específico. Una vez que recibe una solicitud, la procesa y responde al cliente.
- **Cliente:** Es el proceso que inicia la comunicación. Se conecta al servidor, envía una petición, recibe la respuesta y finalmente cierra la conexión.

En esta práctica, además de implementar el modelo cliente-servidor básico, se abordó la necesidad de **multiprocesamiento**, es decir, la capacidad del servidor para atender a varios clientes **de manera concurrente**. Para ello, se utilizaron **hilos (threads)** en Java, de modo que cada cliente conectado es atendido por un hilo independiente, evitando que un cliente bloquee la atención de otros.

Este tipo de arquitectura es la base de sistemas distribuidos reales, donde miles de usuarios pueden conectarse a un mismo servidor de manera simultánea.

2. Problemática

El reto principal era construir un sistema en Java capaz de:

1. **Establecer un servidor TCP** que escuche en un puerto específico y acepte conexiones de múltiples clientes.
2. **Implementar un cliente TCP** que se conecte al servidor, envíe mensajes y reciba respuestas.
3. **Garantizar la concurrencia** en el servidor, de manera que la llegada de múltiples clientes no provoque bloqueos o interrupciones en la comunicación.

4. **Manejar los flujos de entrada/salida (I/O)** de forma eficiente, asegurando que los datos enviados por los clientes fueran recibidos correctamente y que las respuestas fueran transmitidas sin errores.

En sistemas distribuidos reales, si el servidor no maneja adecuadamente los hilos, pueden presentarse problemas como:

- **Bloqueo del servidor:** si un cliente tarda demasiado en enviar información, el servidor queda esperando indefinidamente.
- **Condiciones de carrera:** si varios clientes comparten recursos sin sincronización adecuada, los datos pueden corromperse.
- **Desbordamiento de conexiones:** si no se controlan los hilos, el servidor puede quedar sobrecargado.

3. Solución

La solución se desarrolló utilizando la API de **sockets en Java** y el manejo de **hilos**. La arquitectura implementada fue la siguiente:

- **Servidor (Servidor.java)**
 - Crea un ServerSocket en el puerto definido.
 - Se queda en estado de escucha mediante accept().
 - Por cada cliente que se conecta, lanza un nuevo hilo (ManejadorCliente) para gestionar la comunicación.
 - Esto permite que el servidor pueda atender múltiples clientes simultáneamente.
- **ManejadorCliente (ManejadorCliente.java)**
 - Es una clase que implementa la interfaz Runnable.
 - Cada vez que un cliente se conecta, se crea un nuevo objeto ManejadorCliente y se ejecuta en un hilo independiente.
 - El hilo se encarga de recibir los mensajes del cliente, procesarlos y devolver una respuesta.
 - Al finalizar la comunicación, cierra la conexión.
- **Cliente (Cliente.java)**
 - Se conecta al servidor usando Socket(IP, Puerto).
 - Envía datos con write() y recibe respuesta con read().

- Finalmente, libera los recursos cerrando el socket.

Este diseño cumple con los requisitos de concurrencia, comunicación confiable y simplicidad de implementación.

4. Implementación

La práctica se implementó en Java, utilizando tres clases principales:

1. Servidor.java

- Inicializa un ServerSocket en un puerto definido (ejemplo: 8080).
- Se mantiene en un bucle infinito esperando conexiones de clientes.
- Cada vez que un cliente se conecta, ejecuta un nuevo hilo con un ManejadorCliente.

```
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;

public class Servidor {
    Run | Debug
    public static void main(String[] args) {
        // Puerto en el que el servidor escuchará.
        final int PUERTO = 8080;

        try (ServerSocket serverSocket = new ServerSocket(PUERTO)) {
            System.out.println("✅ Servidor iniciado y escuchando en el puerto " + PUERTO);

            // Bucle infinito para aceptar conexiones de clientes continuamente.
            while (true) {
                System.out.println("Esperando a un cliente...");

                // 1. accept() - Bloquea la ejecución hasta que un cliente se conecte.
                Socket socketCliente = serverSocket.accept();
                System.out.println("Cliente conectado desde " + socketCliente.getInetAddress().getHostAddress() + " ");

                // 2. Por cada cliente, crea un nuevo hilo para manejar la comunicación.
                ManejadorCliente manejador = new ManejadorCliente(socketCliente);
                Thread hiloCliente = new Thread(manejador);
                hiloCliente.start();
            }
        } catch (IOException e) {
            System.err.println("❌ Error en el servidor: " + e.getMessage());
        }
    }
}
```

2. ManejadorCliente.java

- Clase encargada de la comunicación con el cliente.
- Recibe los datos enviados y responde con un mensaje de confirmación.
- Está diseñada para ejecutarse de forma independiente gracias a Runnable.

```

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.Socket;

// Runnable permite que esta clase se ejecute en un hilo separado.
public class ManejadorCliente implements Runnable {
    private final Socket socketCliente;

    public ManejadorCliente(Socket socket) {
        this.socketCliente = socket;
    }

    @Override
    public void run() {
        // Usamos try-with-resources para asegurar que todo se cierre automáticamente.
        try (
            PrintWriter salida = new PrintWriter(socketCliente.getOutputStream(), autoFlush:true);
            BufferedReader entrada = new BufferedReader(new InputStreamReader(socketCliente.getInputStream()))
        ) {
            String mensajeCliente;

            // 3. read() - Lee los mensajes que envía el cliente.
            while ((mensajeCliente = entrada.readLine()) != null) {
                System.out.println("Mensaje recibido del cliente: " + mensajeCliente);

                // 4. Lógica de negocio: procesamos el mensaje.
                String respuesta = mensajeCliente.toUpperCase();

                // 5. write() - Enviamos la respuesta de vuelta al cliente.
                salida.println("Respuesta del servidor: " + respuesta);

                // Si el cliente envía "adios", terminamos la conexión.
                if (mensajeCliente.equalsIgnoreCase("adios")) {
                    break;
                }
            }
        } catch (IOException e) {
            System.err.println("Error al manejar el cliente: " + e.getMessage());
        } finally {
            try {
                // 6. close() - Cerramos el socket del cliente.
                socketCliente.close();
                System.out.println(x:"Conexión con el cliente cerrada.");
            } catch (IOException e) {
                // Ignorar
            }
        }
    }
}

```

3. Cliente.java

- Establece la conexión con el servidor.
- Envía un mensaje de prueba y espera la respuesta.

- Cierra la conexión al finalizar.

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.Socket;
import java.util.Scanner;

public class Cliente {
    Run | Debug
    public static void main(String[] args) {
        final String HOST = "localhost";
        final int PUERTO = 6000;

        try {
            // 1. socket() y connect() - Crea el socket y se conecta al servidor.
            Socket socket = new Socket(HOST, PUERTO);
            PrintWriter salida = new PrintWriter(socket.getOutputStream(), true);
            BufferedReader entrada = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            Scanner scanner = new Scanner(System.in)

        } {
            System.out.println(x: "✅ Conectado al servidor. Escribe 'adios' para salir.");
            String linea;

            do {
                System.out.print(s: "Escribe un mensaje: ");
                linea = scanner.nextLine();

                // 2. write() - Envía el mensaje al servidor.
                salida.println(linea);

                // 3. read() - Lee la respuesta del servidor y la muestra.
                String respuestaServidor = entrada.readLine();
                System.out.println(respuestaServidor);

            } while (!linea.equalsIgnoreCase(anotherString: "adios"));

        } catch (IOException e) {
            System.err.println("❌ Error en el cliente: " + e.getMessage());
        }
    }
}
```

Este enfoque modular facilita la comprensión y depuración del sistema.

5. Resultados

Al ejecutar la aplicación se observaron los siguientes resultados:

- El **servidor** inició correctamente, mostrando en consola el mensaje de que estaba escuchando en el puerto definido.
- Se pudieron conectar **múltiples clientes de manera concurrente**. Cada cliente recibía respuesta del servidor de forma independiente.
- Al desconectar un cliente, el servidor permanecía disponible para aceptar nuevas conexiones.
- La comunicación fue confiable: todos los mensajes enviados fueron recibidos en el orden correcto y sin pérdida de información.

Esto demostró que el uso de hilos en el servidor permitió manejar múltiples clientes al mismo tiempo, garantizando la escalabilidad del sistema.

```
Microsoft Windows [Version 10.0.26100.4349]
(c) Microsoft Corporation. All rights reserved.

C:\Users\miaro>cd documents
C:\Users\miaro\Documents>cd sistemas distribuidos
C:\Users\miaro\Documents\Sistemas distribuidos>cd practica 2
C:\Users\miaro\Documents\Sistemas distribuidos\Practica 2>java
va Cliente
? Conectado al servidor. Escribe 'adios' para salir.
Escribe un mensaje: hola desde cliente 2
Respuesta del servidor: HOLA DESDE CLIENTE 2
Escribe un mensaje:

Error: Could not find or load main class servidor
Caused by: java.lang.NoClassDefFoundError: servidor (wrong nam
e: Servidor)

C:\Users\miaro\Documents\Sistemas distribuidos\Practica 2>java
Servidor
? Servidor iniciado y escuchando en el puerto 6000
Esperando a un cliente...
¡Cliente conectado desde 127.0.0.1!
Esperando a un cliente...
¡Cliente conectado desde 127.0.0.1!
Esperando a un cliente...
Mensaje recibido del cliente: hola desde cliente1
Mensaje recibido del cliente: hola desde cliente 2

C:\Users\miaro\Documents\Sistemas distribuidos\Practica 2>java Cliente
? Conectado al servidor. Escribe 'adios' para salir.
Escribe un mensaje: hola desde cliente1
Respuesta del servidor: HOLA DESDE CLIENTE1
Escribe un mensaje:

C:\Users\miaro\Documents\Sistemas distribuidos\Practica 2>java
Servidor
? Servidor iniciado y escuchando en el puerto 6000
Esperando a un cliente...
¡Cliente conectado desde 127.0.0.1!
Esperando a un cliente...
¡Cliente conectado desde 127.0.0.1!
Esperando a un cliente...
Mensaje recibido del cliente: hola desde cliente1
Mensaje recibido del cliente: hola desde cliente 2
Error al manejar el cliente: Connection reset
Conexión con el cliente cerrada.
Mensaje recibido del cliente: adios
Conexión con el cliente cerrada.
```

6. Conclusión

La práctica permitió comprender y aplicar los conceptos de **sockets TCP**, **multiprocesamiento** y **modelo cliente-servidor** en Java.

Las principales conclusiones son:

1. El modelo cliente-servidor es la base de la mayoría de los sistemas distribuidos actuales.
2. Los **sockets TCP** permiten establecer una comunicación confiable y bidireccional entre procesos distribuidos en diferentes máquinas.
3. La concurrencia en el servidor, lograda mediante **hilos**, es esencial para poder atender múltiples clientes simultáneamente sin bloqueo.

4. La correcta gestión de los sockets (abrir, utilizar y cerrar) asegura que los recursos del sistema se usen eficientemente.
5. Este ejercicio representa una introducción práctica a sistemas más complejos como servidores web, aplicaciones de mensajería instantánea, videoconferencias y sistemas de bases de datos distribuidas.

7. Referencias

- Oracle. (n.d.). *Lesson: All About Sockets (The Java™ Tutorials > Custom Networking)*. Recuperado de:
<https://docs.oracle.com/javase/tutorial/networking/sockets/>
- GeeksforGeeks. (n.d.). *Socket Programming in Java*. Recuperado de:
<https://www.geeksforgeeks.org/socket-programming-in-java/>
- Baeldung. (n.d.). *A Guide to Java Sockets*. Recuperado de:
<https://www.baeldung.com/a-guide-to-java-sockets>